CONOCIMIENTO Y RAZONAMIENTO AUTOMATIZADO

PRÁCTICA 1: MONOPOLY

DIEGO DÍAZ VIDAL
FERNANDO CEMBELLÍN SECO
GRADO EN INGENIERÍA INFORMÁTICA
CURSO 2025-2026

Resumen

La programación lógica constituye un paradigma declarativo en el que la resolución de problemas se basa en la definición de hechos, reglas y relaciones lógicas, en lugar de en la descripción explícita de una secuencia de instrucciones imperativas. Dentro de este enfoque, Prolog destaca por su capacidad para representar conocimiento simbólico, modelar estados y aplicar reglas de inferencia sobre estructuras de datos simples, especialmente listas. Estas características lo convierten en una herramienta especialmente adecuada para abordar problemas donde el interés principal no reside tanto en la interfaz o en la eficiencia de ejecución, sino en la correcta formalización del dominio y en la transformación progresiva de estados mediante reglas bien definidas.

En esta práctica se ha utilizado el juego Monopoly como caso de estudio para aprender y aplicar los fundamentos de la programación lógica. El juego actúa como un contexto suficientemente rico para modelar tableros, jugadores, turnos, recursos económicos y reglas encadenadas, permitiendo trabajar de forma natural con listas, estructuras y predicados de transformación de estado. El desarrollo se ha organizado siguiendo una metodología incremental basada en épicas: en cada una de ellas se han definido unos objetivos concretos, se ha implementado la funcionalidad correspondiente en Prolog y, finalmente, se han obtenido resultados a partir de escenarios de prueba, métricas y simulaciones. Aunque el Monopoly sirve como hilo conductor de la práctica, el verdadero objetivo del trabajo ha sido comprender el paradigma lógico y desarrollar habilidades de representación simbólica y manipulación de listas en Prolog.

Contenido

Resumen 2
Introducción 5
Metodología de trabajo 5
1. Épica 1 – Tablero 6
1.1. Objetivos 6
1.2. Desarrollo 6
1.3. Resultados y análisis 6
2. Épica 2 – Movimiento de dados 7
2.1. Objetivos 7
2.2. Desarrollo 7
2.3. Resultados y análisis 8
3. Épica 3 – Compras 8
3.1. Objetivos 8
3.2. Desarrollo 9
3.3. Resultados y análisis 9
4. Épica 4 – Alquileres 10
4.1. Objetivos 10
4.2. Desarrollo 10
4.3. Resultados y análisis 10
5. Épica 5 – Cárcel 11
5.1. Objetivos 11
5.2. Desarrollo 11
5.3. Resultados y análisis 11
6. Épica 6 – Bancarrota 12
6.1. Objetivos 12
6.2. Desarrollo 12
6.3. Resultados y análisis 13
7. Épica 7 – Parking 13
7.1. Objetivos 13
7.2. Desarrollo 13
7.3. Resultados y análisis 14

8. Épica 8 – Monopoly 14
8.1. Objetivos..... 14
8.2. Desarrollo 15
8.3. Resultados y análisis 15
Conclusiones..... 15
Bibliografía 16
Anexos 17

Introducción

La presente práctica tiene como finalidad introducir y consolidar el uso de la programación lógica mediante el desarrollo progresivo de una simulación del juego Monopoly en Prolog. Más que reproducir una versión completa del juego con todos sus detalles comerciales, el propósito principal ha sido emplear este dominio como excusa didáctica para trabajar la representación de conocimiento, la manipulación de listas y la aplicación coordinada de reglas sobre estados dinámicos. En este sentido, el interés del trabajo no ha estado únicamente en el resultado final, sino en el proceso de modelado lógico de los distintos elementos del problema: tablero, jugadores, movimientos, compras, alquileres, cárcel, bancarrota, parking y monopolios.

Para ello, el desarrollo se ha estructurado en varias épicas, cada una centrada en una parte concreta de la funcionalidad global. Este enfoque ha permitido avanzar de forma incremental, validando cada bloque antes de integrarlo con el resto del sistema. A lo largo de la memoria se describen los objetivos de cada etapa, las decisiones de representación adoptadas, la implementación realizada y los resultados obtenidos, con el fin de mostrar cómo un problema aparentemente lúdico puede convertirse en un ejercicio completo de aprendizaje del paradigma lógico.

Metodología de trabajo

Para el desarrollo de la práctica se ha seguido un enfoque colaborativo apoyado en distintas herramientas y una organización iterativa del trabajo. Se ha utilizado un repositorio de GitHub para gestionar el código fuente, donde se han ido incorporando los avances de forma progresiva, organizados por épicas. Para la comunicación entre los miembros del equipo se ha empleado Discord, mientras que la redacción de la memoria y la toma de apuntes se ha realizado mediante un documento compartido en Microsoft OneDrive. Además, se han llevado a cabo reuniones presenciales semanales o bisemanales, con el objetivo de trabajar de forma conjunta, resolver dudas y poner en común el progreso individual realizado durante la semana.

Inicialmente, el enfoque de la práctica no fue el más adecuado. En una primera sesión de trabajo conjunta, larga e intensiva, se intentó desarrollar una versión completa del juego Monopoly de forma directa. Sin embargo, este planteamiento resultó ser un error, ya que todavía no se había tenido la oportunidad de contrastar ideas con el profesor ni de recibir su feedback. Tras una reunión posterior con el profesor, se replanteó completamente la metodología de trabajo, adoptando un enfoque incremental basado en épicas, así como en la obtención de métricas a partir de escenarios de prueba, aspecto en el que se hizo especial énfasis.

A partir de ese momento, el desarrollo se estructuró en distintas épicas, cada una centrada en una funcionalidad concreta del sistema. Cada épica partía de la anterior, ampliando progresivamente el modelo del juego y asegurando en todo momento la coherencia global del sistema. Para validar cada nueva funcionalidad, se definieron escenarios de prueba específicos, que permitían comprobar de forma controlada el comportamiento del sistema y analizar los resultados obtenidos mediante métricas. Este enfoque ha permitido no solo garantizar la correcta implementación de cada parte, sino también entender en profundidad la evolución del estado del sistema a lo largo de las simulaciones.

1. Épica 1 – Tablero

1.1. Objetivos

El objetivo de esta épica es definir la representación del tablero y del estado inicial del juego en Prolog, estableciendo una base sólida para el resto del desarrollo. A nivel de juego, se busca modelar correctamente las casillas del Monopoly (propiedades, estaciones, impuestos, etc.) y sus características. A nivel de código, el objetivo es diseñar estructuras de datos claras y coherentes con el paradigma lógico, que permitan consultar y manipular el estado del juego de forma sencilla.

1.2. Desarrollo

El tablero se ha implementado como una lista de estructuras casilla/5, lo que permite trabajar de forma declarativa mediante recorridos y filtros. Se han definido los distintos tipos de casillas y colores de propiedades, así como predicados auxiliares para consultar el tablero (acceso por posición, comprobación de comprables, etc.).

Además, se ha modelado el estado inicial del juego mediante estructuras para el banco, los jugadores y la partida, generando automáticamente los activos del banco a partir del tablero. También se han incluido validaciones del tablero para asegurar su coherencia (tamaño, índices consecutivos y casillas obligatorias).

Por último, se ha separado la lógica en módulos (tablero y métricas), facilitando la organización del código y permitiendo exportar información estructural del tablero a CSV.

1.3. Resultados y análisis

En esta épica no se evalúa comportamiento dinámico, sino la correcta definición del modelo. Para ello, se han creado escenarios de prueba centrados en validar el tablero y las estructuras iniciales, comprobando que las casillas están correctamente definidas y que el estado inicial de la partida se genera de forma consistente.

Asimismo, se han utilizado métricas estructurales (exportadas a CSV y visualizadas mediante scripts en Python) para verificar la distribución de casillas y propiedades, lo que permite validar indirectamente que el modelo representa correctamente el tablero del juego.

A continuación, se muestra el contenido de dos tablas generadas para validar la correcta implementación del tablero.

Gráfica tipos de casillas

2. Épica 2 – Movimiento de dados

2.1. Objetivos

El objetivo de esta épica es incorporar el movimiento de los jugadores a lo largo del tablero a partir del lanzamiento de dos dados. Para ello, se busca actualizar correctamente la posición de cada jugador, respetar el carácter circular del tablero y detectar cuándo se pasa por la casilla de salida. Además, esta etapa pretende introducir una simulación reproducible, de forma que los resultados puedan analizarse y compararse entre distintos escenarios.

2.2. Desarrollo

La funcionalidad se ha dividido en varios módulos: dados, movimiento, escenarios y métricas.

En dados.pl se ha implementado un sistema de pseudoaleatoriedad reproducible. En lugar de usar un generador aleatorio, se utiliza una secuencia fija de valores entre 1 y 6. A partir de una semilla, esta secuencia se rota, de forma que cada escenario empieza en un punto distinto. Cada tirada consume los dos siguientes valores de la lista y, cuando se agota, se vuelve a reutilizar. Esto permite simular tiradas variadas pero, al mismo tiempo, repetir exactamente una ejecución si se usa la misma semilla, cosa que se probó con el profesor en clase.

El movimiento se gestiona en movimiento.pl. En cada turno se selecciona al jugador correspondiente, se lanzan los dados, se calcula la suma y se actualiza la posición usando aritmética modular para mantener el recorrido circular del tablero. También se detecta si el jugador pasa por salida.

Cada turno genera un evento con toda la información relevante (dados, posiciones, casilla, etc.), lo que permite guardar el historial completo de la partida. A partir de estos eventos, en metricas.pl se calculan estadísticas como la cobertura del tablero o la frecuencia de las tiradas.

2.3. Resultados y análisis

Los resultados obtenidos permiten comprobar que el sistema mueve correctamente a los jugadores, mantiene el orden de turnos y registra de forma adecuada los eventos de cada jugada. Los escenarios más largos muestran además que el tablero se recorre de manera progresiva y que la cobertura aumenta con el número de turnos simulados.

Uno de los aspectos más relevantes de esta épica es el uso de una pseudoaleatoriedad controlada. Aunque no se trata de azar real, esta solución resulta muy útil en el contexto de la práctica, ya que permite repetir exactamente una simulación, comparar escenarios con distintas semillas y analizar los resultados de forma consistente. En consecuencia, esta épica no solo añade movimiento al juego, sino que también establece una base sólida para estudiar el comportamiento del sistema en las siguientes fases

Comparación de frecuencia de visitas con simulación corta (2 jugadores, 100 turnos)

Comparación de frecuencia de visitas con simulación larga (6 jugadores, 2000 turnos)

Frecuencia de paso por salida por jugador con simulación larga (6 jugadores, 2000 turnos)

Frecuencia de sumas de dados en simulación corta (4 jugadores, 100 turnos)

Frecuencia de sumas de dados en simulación larga (8 jugadores, 1000 turnos)

3. Épica 3 – Compras

3.1. Objetivos

El objetivo de esta épica es incorporar la mecánica de compra de propiedades al modelo del juego. En concreto, se busca que un jugador pueda adquirir automáticamente una propiedad o estación cuando cae en una casilla comprable, siempre que disponga de dinero suficiente. Esta decisión se tomó tras comentarlo con el profesor, ya que resulta la opción más adecuada para las simulaciones: si una propiedad puede comprarse, se compra, eliminando así la posibilidad de “pasar” y simplificando el análisis del comportamiento del sistema.

A nivel de programación lógica, esta etapa pretende ampliar la transformación del estado de la partida, introduciendo modificaciones tanto en el jugador (dinero y propiedades) como en el banco. Además, se busca registrar estos cambios mediante eventos que permitan su posterior análisis.

3.2. Desarrollo

La lógica de compras se ha implementado en el módulo compras.pl, mediante un predicado que decide si se realiza la compra en función de la casilla y del estado del jugador.

En primer lugar, se comprueba si la casilla es comprable. Si no lo es, no se realiza ninguna acción. En caso contrario, se verifica si el activo sigue disponible en el banco y si el jugador tiene suficiente dinero.

Se ha definido una política sencilla: comprar siempre que haya fondos suficientes. Cuando se cumple esta condición, el activo se elimina del banco, se añade a la lista de propiedades del jugador y se actualiza su dinero restando el precio. Si no se puede comprar, el estado permanece sin cambios.

Además, cuando se realiza una compra, se genera un evento que recoge información relevante como el turno, el jugador, la propiedad adquirida y el dinero antes y después de la operación. Esto permite integrar la compra dentro del historial de la partida y facilita el análisis posterior.

3.3. Resultados y análisis

Los resultados obtenidos muestran que el sistema gestiona correctamente la adquisición de propiedades. En los escenarios de prueba, los jugadores compran automáticamente cuando caen en casillas disponibles y tienen suficiente dinero, y dejan de hacerlo cuando no pueden permitírselo o cuando la propiedad ya ha sido adquirida por otro jugador.

El uso de una política de compra simple permite validar fácilmente el comportamiento del sistema y comprobar que las transformaciones de estado (jugador y banco) se realizan de forma coherente. Además, el registro de eventos de compra facilita el seguimiento de la evolución de la partida y prepara el sistema para futuras funcionalidades, como el cálculo de alquileres o la detección de monopolios.

En conjunto, esta épica añade un componente económico básico al modelo y marca la transición desde una simulación puramente de movimiento a una simulación con interacción entre jugadores y recursos.

Evolución de compras acumuladas a lo largo de la simulación

Propiedades finales por jugador

4. Épica 4 – Alquileres

4.1. Objetivos

El objetivo de esta épica es incorporar la mecánica de alquiler al modelo del juego. En concreto, se busca que cuando un jugador caiga en una propiedad o estación que pertenece a otro jugador, se realice automáticamente el pago correspondiente. Con ello se introduce una interacción económica directa entre jugadores, avanzando hacia una simulación más completa del Monopoly.

4.2. Desarrollo

La lógica de alquileres se ha implementado en el módulo alquileres.pl. El sistema comprueba primero si en ese turno se acaba de realizar una compra, ya que en ese caso no procede cobrar alquiler. También se verifica que la casilla en la que ha caído el jugador sea comprable.

Si la casilla puede generar alquiler, se busca entre el resto de jugadores si alguno es propietario de ese activo. Cuando existe dueño, se calcula un alquiler teórico como una quinta parte del precio de la casilla. A continuación, el jugador que cae en ella paga esa cantidad, o todo el dinero que le quede si no alcanza para cubrir el alquiler completo, y el propietario recibe dicho importe.

Además, se genera un evento de alquiler con información como el pagador, el propietario, la casilla, el importe teórico y el pago real realizado. Así, esta acción queda registrada dentro del historial de la partida para su posterior análisis.

4.3. Resultados y análisis

Los resultados muestran que el sistema identifica correctamente cuándo una casilla debe generar alquiler y actualiza de forma coherente el dinero de ambos jugadores. También se contempla el caso en el que el jugador no dispone de fondos suficientes, realizando un pago parcial con el dinero restante.

La decisión de calcular el alquiler como una fracción fija del precio simplifica esta primera versión de la mecánica y permite centrar el análisis en la correcta transformación del estado. En conjunto, esta épica añade una nueva capa económica al modelo y deja preparada la base para etapas posteriores, como la bancarrota o la mejora de propiedades.

Importe acumulado de alquileres

Evolución del dinero por jugador a lo largo de la partida

5. Épica 5 – Cárcel

5.1. Objetivos

El objetivo de esta épica es incorporar la mecánica de la cárcel al modelo del juego. En concreto, se busca gestionar el estado de los jugadores cuando son enviados a la cárcel, limitando su capacidad de movimiento y definiendo las condiciones bajo las cuales pueden salir.

Además, se pretende introducir decisiones dentro del sistema, permitiendo que el comportamiento del jugador en la cárcel dependa de una política definida, lo que añade mayor flexibilidad al modelo.

Es importante diferenciar la casilla de ir a la cárcel de la de visita, ya que su comportamiento es distinto dentro del modelo.

5.2. Desarrollo

La lógica de la cárcel se ha implementado en el módulo carcel.pl. Cuando un jugador entra en la cárcel, su posición se fija y su estado cambia a encarcelado(N), donde N representa el número de intentos realizados para salir.

En cada turno, si el jugador está en la cárcel, no sigue el flujo normal del juego, sino que se aplica una lógica específica. Se han definido distintas políticas de decisión: pagar siempre, tirar siempre o una política mixta basada en un umbral de dinero.

Si el jugador decide pagar (y dispone de dinero suficiente), sale de la cárcel inmediatamente. Si decide tirar los dados, solo saldrá si obtiene dobles; en caso contrario, aumenta su contador de intentos y pierde el turno. Tras tres intentos fallidos, el jugador sale automáticamente de la cárcel.

Durante todo este proceso se generan eventos que registran la entrada en la cárcel, los intentos de salida, los fallos y las liberaciones, lo que permite analizar posteriormente el comportamiento del sistema.

5.3. Resultados y análisis

En los escenarios de prueba se ha podido comprobar que la lógica de la cárcel funciona de forma coherente a lo largo de varios turnos. En concreto, se ha simulado el comportamiento de jugadores que permanecen en la cárcel durante varios turnos consecutivos, observando cómo en cada uno de ellos se aplica la decisión correspondiente según la política definida.

Se han dado casos en los que los jugadores salen pagando directamente, mientras que en otros permanecen varios turnos intentando salir mediante tiradas de dados. En estos

últimos, se ha podido comprobar que el sistema registra correctamente los intentos y que el jugador solo sale cuando obtiene dobles o cuando alcanza el límite de intentos, momento en el que se libera automáticamente.

Esto permite ver claramente la evolución del estado del jugador turno a turno, diferenciando cuándo pierde el turno y cuándo vuelve a participar con normalidad en la partida. Además, al tratarse de una simulación con pseudoaleatoriedad, es posible repetir exactamente los mismos casos y analizar con detalle en qué turno se produce la salida de la cárcel y bajo qué condición (pago o azar).

En conjunto, esta épica introduce un comportamiento más realista en el flujo del juego y añade variabilidad en las partidas, ya que no todos los jugadores permanecen el mismo número de turnos en la cárcel ni salen de la misma forma.

Salida cárcel por liberación automática

Salida cárcel por pago

Salida por números dobles

6. Épica 6 – Bancarrota

6.1. Objetivos

El objetivo de esta épica es gestionar las situaciones en las que un jugador se queda sin dinero, incorporando la lógica de bancarrota al modelo del juego. Se busca decidir si el jugador puede recuperarse vendiendo sus propiedades o si, por el contrario, debe ser eliminado de la partida.

6.2. Desarrollo

La lógica se ha implementado en bancarrota.pl. Cada vez que un jugador queda con dinero negativo, se activa este proceso.

Primero se calcula el valor de liquidación de sus propiedades, considerando que cada una se vende por la mitad de su precio. Estas propiedades vuelven al banco y el dinero obtenido se suma al saldo del jugador.

A partir de ahí pueden ocurrir dos cosas: si el jugador consigue volver a tener dinero positivo o cero, continúa en la partida pero sin propiedades; si sigue en negativo, se elimina directamente de la lista de jugadores.

Además, se genera un evento de bancarrota donde se recoge información como el turno, la causa (por ejemplo, un alquiler), el dinero antes y después, y si el jugador sobrevive o es eliminado.

6.3. Resultados y análisis

En las simulaciones se han observado varios casos de jugadores que entran en bancarrota tras pagar alquileres elevados. En algunos turnos, el jugador consigue recuperarse gracias a la liquidación de sus propiedades, continuando la partida aunque en una posición más débil. En otros casos, el dinero obtenido no es suficiente y el jugador acaba siendo eliminado.

Esto se puede seguir fácilmente en los eventos, donde se ve en qué turno ocurre la bancarrota y cuál es el resultado. También se aprecia cómo, a medida que avanza la partida, aumentan los casos en los que los jugadores no pueden recuperarse, ya que van perdiendo propiedades progresivamente.

En conjunto, esta épica añade una condición importante al juego, ya que introduce la posibilidad de eliminación de jugadores y hace que la simulación evolucione hacia situaciones más realistas donde el número de participantes va disminuyendo con el tiempo.

Orden de bancarrotas en la partida (10 jugadores, 500 turnos, semilla 11)

7. Épica 7 – Parking

7.1. Objetivos

El objetivo de esta épica es implementar la mecánica del parking gratuito, permitiendo que los jugadores reciban una recompensa económica al caer en esta casilla. Además, se busca conectar esta mecánica con los impuestos del juego, de forma que el dinero que se pierde en determinadas casillas pueda reutilizarse dentro de la simulación.

7.2. Desarrollo

a lógica se ha implementado en el módulo parking.pl, donde se define el comportamiento cuando un jugador cae en la casilla de parking. En ese caso, el jugador recibe automáticamente una cantidad de dinero procedente de una caja acumulada en el banco.

En esta implementación se ha decidido que el jugador reciba la mitad del dinero disponible en dicha caja. Tras el cobro, el banco actualiza el valor restante y el jugador incrementa su dinero en la cantidad obtenida.

Por otro lado, los impuestos, implementados en impuestos.pl, reducen el dinero de los jugadores cuando caen en ciertas casillas. Aunque su lógica es independiente, conceptualmente estos importes alimentan la caja del parking, estableciendo así una relación entre ambas mecánicas.

Además, se generan eventos tanto de impuestos como de parking, lo que permite registrar cuándo se pierde dinero y cuándo se redistribuye, facilitando su análisis dentro de la simulación.

7.3. Resultados y análisis

En los escenarios simulados se observa que el parking actúa como un mecanismo de redistribución de dinero. Parte del dinero que los jugadores pierden a lo largo de la partida, especialmente a través de los impuestos, vuelve al sistema mediante esta casilla.

Esta mecánica se decidió incluir tras comentarlo con el profesor, con el objetivo de darle más juego a las simulaciones. De esta forma, no todo depende únicamente de propiedades y alquileres, sino que se introduce un elemento adicional que puede cambiar la evolución económica de los jugadores.

También se aprecia que el premio del parking no es constante, ya que depende del dinero acumulado previamente. Esto añade variabilidad a las partidas, haciendo que algunos jugadores puedan recuperarse en determinados turnos mientras que otros no se benefician en absoluto.

En conjunto, esta épica aporta un extra al modelo que hace las simulaciones más dinámicas e interesantes, conectando la pérdida de dinero (impuestos) con su posterior redistribución.

Evolución de la caja de parking tras cobros

Evolución de la caja de parking a lo largo de la simulación

8. Épica 8 – Monopoly

Comprar casas

8.1. Objetivos

El objetivo de esta épica es incorporar la detección de monopolios y la compra automática de casas al modelo del juego. En concreto, se busca identificar cuándo un jugador posee todas las propiedades de un mismo color y, a partir de ese momento, permitir que mejore esas propiedades comprando casas.

Con esta mecánica se añade una nueva capa estratégica al sistema, ya que los jugadores no solo pueden comprar propiedades, sino también reforzarlas para aumentar su valor dentro de la partida.

8.2. Desarrollo

La lógica se ha implementado en el módulo monopoly.pl. El sistema comprueba primero si el jugador tiene algún monopolio, es decir, si posee todas las propiedades de un mismo color. Para ello se revisan los distintos grupos de colores del tablero y se detectan aquellos en los que el jugador reúne todas las propiedades necesarias.

Cuando un jugador tiene al menos un monopolio, se realiza automáticamente una ronda de compra de casas. En esta implementación se ha optado por comprar una casa en cada propiedad del color monopolizado, siempre que el jugador tenga dinero suficiente. Cada casa cuesta 100 y una propiedad puede tener como máximo 3 casas.

De esta forma, el sistema recorre las propiedades del jugador y va actualizando tanto el número de casas como el dinero disponible. Además, se genera un evento donde se registra el turno, el jugador, los colores en monopolio, el número total de casas compradas y el coste total de la operación.

8.3. Resultados y análisis

En las simulaciones se observa que esta mecánica empieza a tener efecto cuando un jugador consigue reunir todas las propiedades de un color. A partir de ahí, en los turnos correspondientes, el sistema va comprando casas automáticamente, lo que hace que ese jugador refuerce progresivamente su posición.

Se ha podido comprobar que la compra de casas no se realiza de forma arbitraria, sino respetando las condiciones definidas: solo se construye si existe monopolio, si hay dinero suficiente y si la propiedad no ha alcanzado todavía el máximo de casas permitido. Esto hace que el proceso sea fácil de seguir en los eventos de la partida.

Además, esta épica aporta más profundidad a las simulaciones, ya que no todo se reduce a comprar propiedades y pagar alquileres. A medida que avanza la partida, algunos jugadores pueden consolidar su ventaja mediante la construcción de casas, lo que hace que el comportamiento del sistema sea más interesante y cercano a la lógica general del Monopoly.

En conjunto, esta última épica completa el modelo incorporando una mecánica de mejora de propiedades que da más riqueza a la simulación y conecta de forma natural con las épicas anteriores de compras, alquileres y bancarrota.

Compras de casas

Conclusiones

A lo largo de esta práctica se ha trabajado el paradigma de la programación lógica utilizando el Monopoly como ejemplo. Más allá de implementar el juego, lo importante

ha sido aprender a representar el problema en Prolog, definiendo estados y reglas en lugar de seguir un flujo paso a paso como en programación tradicional.

El enfoque por épicas ha sido clave, ya que ha permitido avanzar poco a poco, empezando por algo sencillo como el tablero y llegando a una simulación bastante completa con compras, alquileres, cárcel, bancarrota y mejoras de propiedades. Esto ha ayudado a entender mejor cómo cada parte afecta al resto y cómo evoluciona el estado del sistema turno a turno.

Una de las partes más interesantes ha sido trabajar con simulaciones y métricas. Gracias a la pseudoaleatoriedad, se pueden repetir partidas exactamente iguales, lo que facilita mucho comprobar que todo funciona bien y analizar el comportamiento del sistema con más detalle.

También ha habido dificultades, sobre todo al manejar el estado en Prolog y coordinar todas las reglas que intervienen en cada turno. Aun así, esto ha servido para entender mejor cómo pensar en términos de relaciones y transformaciones, que es la base de la programación lógica.

En general, la práctica ha permitido ver una forma distinta de programar, más centrada en describir el problema que en decir cómo ejecutarlo paso a paso. El Monopoly ha sido un buen contexto para esto, ya que combina varias mecánicas que se pueden modelar de forma progresiva y clara.

Bibliografía

Apuntes de la asignatura *Conocimiento y Razonamiento Automatizado*, curso 2025-2026.

Documentación oficial de SWI-Prolog. Disponible en: https://www.swi-prolog.org

Reglas oficiales del juego Monopoly. Hasbro.

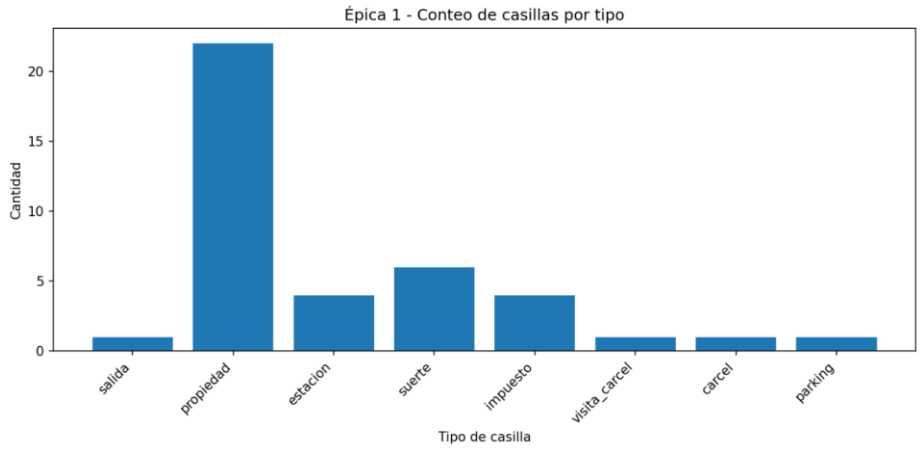
Hasbro (2015). *Monopoly Classic Game – Official Rules*.

Wikipedia. *Monopoly (juego)*. Disponible en: https://es.wikipedia.org/wiki/Monopoly

Uso de herramientas de inteligencia artificial generativa para complementar el trabajo y creación de graficas: ChatGPT (OpenAI) y Gemini (Google).

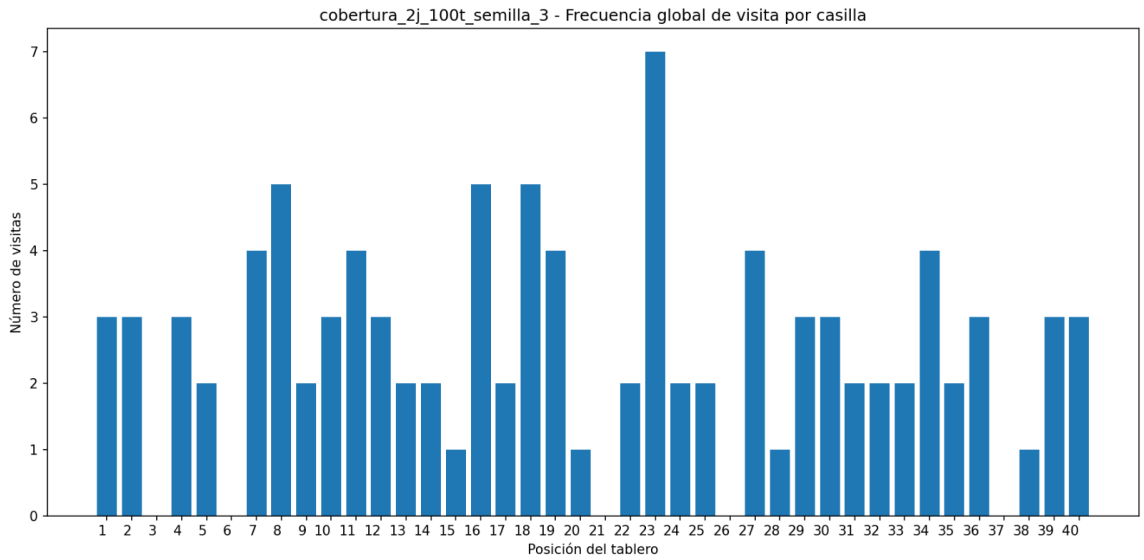
Anexos

Tablero:

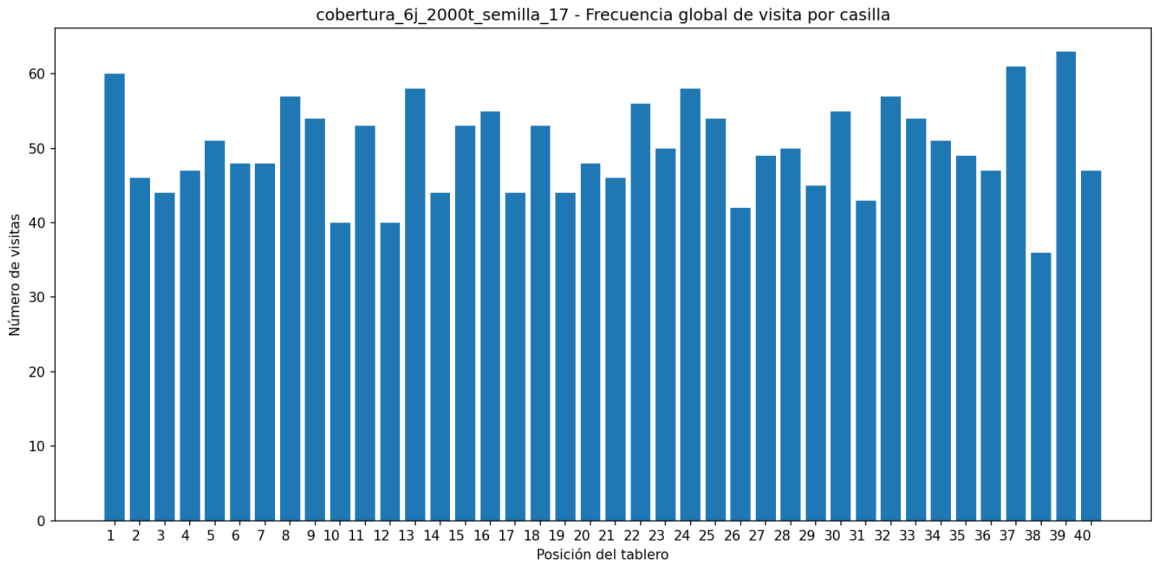


En esta gráfica se muestra el número de casillas de cada tipo dentro del tablero. Más que analizar en detalle la distribución, el objetivo principal es comprobar que el tablero se ha generado correctamente y que el programa funciona como se espera.

Movimiento dados:

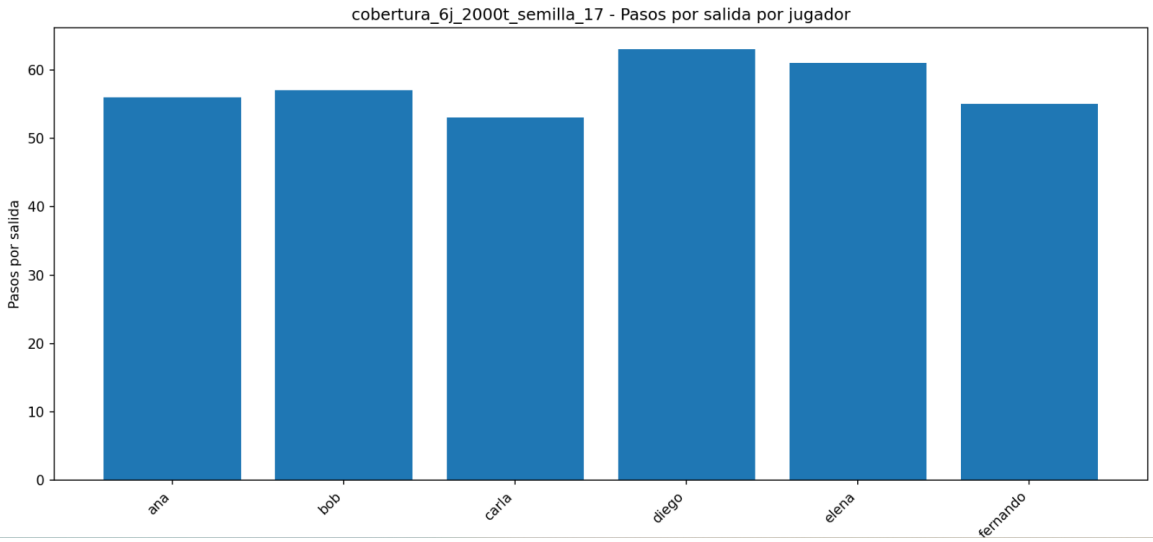


En esta gráfica se muestra la frecuencia de visitas a cada casilla tras la simulación. Se observa que algunas posiciones se visitan más que otras, aunque en general el recorrido es bastante uniforme. Esto indica que el sistema de movimiento funciona correctamente y que, con el paso de los turnos, los jugadores recorren todo el tablero sin quedarse concentrados en zonas concretas.

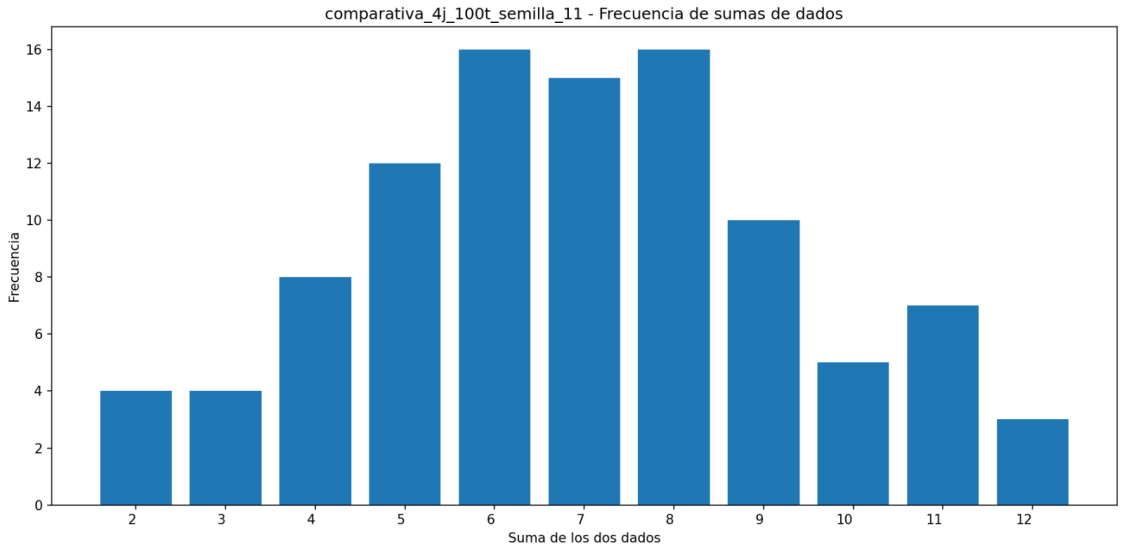


En comparación con la gráfica anterior, en esta simulación con más jugadores y muchos más turnos se observa una distribución mucho más uniforme de las visitas a las casillas. Mientras que en la anterior había más variabilidad y algunas posiciones destacaban más, aquí prácticamente todas las casillas reciben un número de visitas similar.

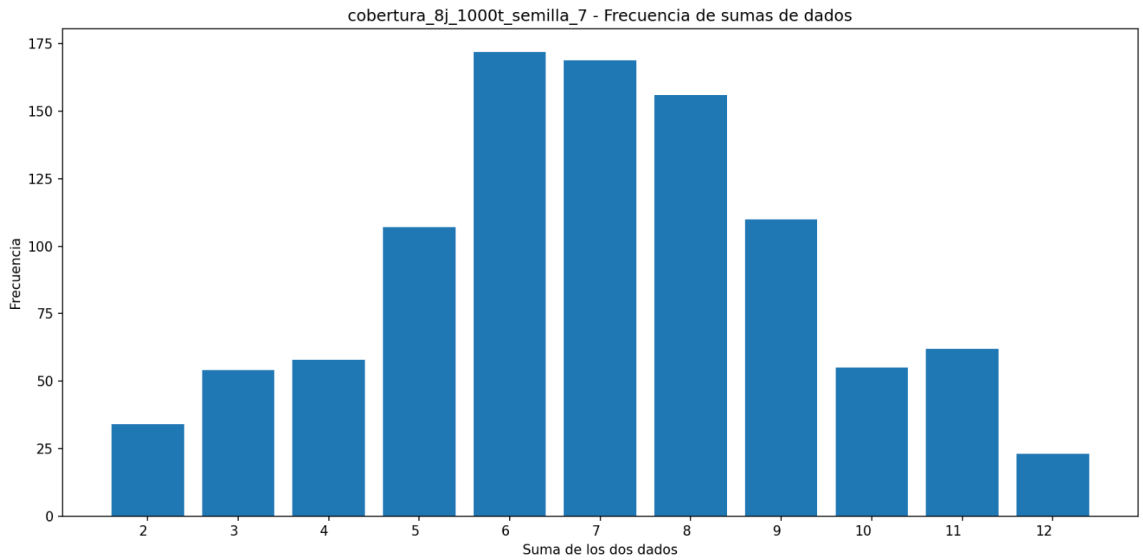
Esto indica que, a medida que aumenta el número de turnos y jugadores, el sistema tiende a estabilizarse y a recorrer el tablero de forma más equilibrada, reduciendo el efecto de la variabilidad inicial.



En esta gráfica se muestra el número de veces que cada jugador pasa por la casilla de salida. Se observa que todos los jugadores tienen valores bastante similares, lo que indica que el sistema de turnos y movimiento es equilibrado y no favorece a ningún jugador en particular.



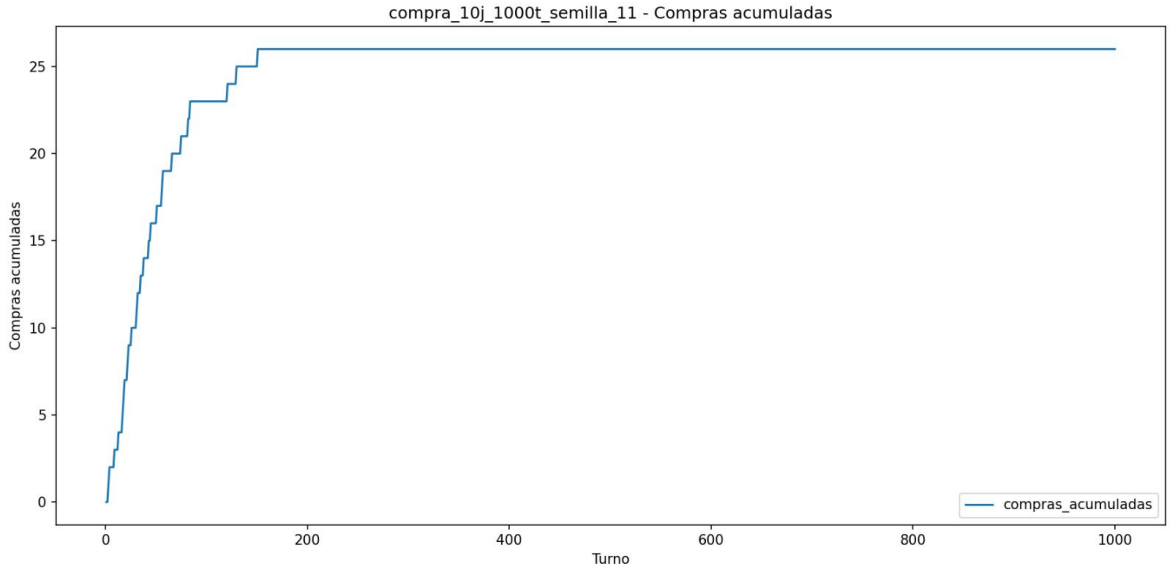
En esta gráfica se muestra la frecuencia de las sumas obtenidas al lanzar los dos dados. Se observa que los valores intermedios (especialmente 6, 7 y 8) aparecen con mayor frecuencia, mientras que los extremos (2 y 12) son menos comunes. Esto coincide con el comportamiento esperado de dos dados y confirma que la pseudoaleatoriedad implementada genera resultados coherentes y realistas dentro de la simulación.



En esta gráfica, al aumentar el número de jugadores y turnos, se observa una distribución aún más clara de las sumas de los dados. Los valores intermedios (6, 7 y 8) destacan claramente como los más frecuentes, mientras que los extremos siguen siendo menos comunes.

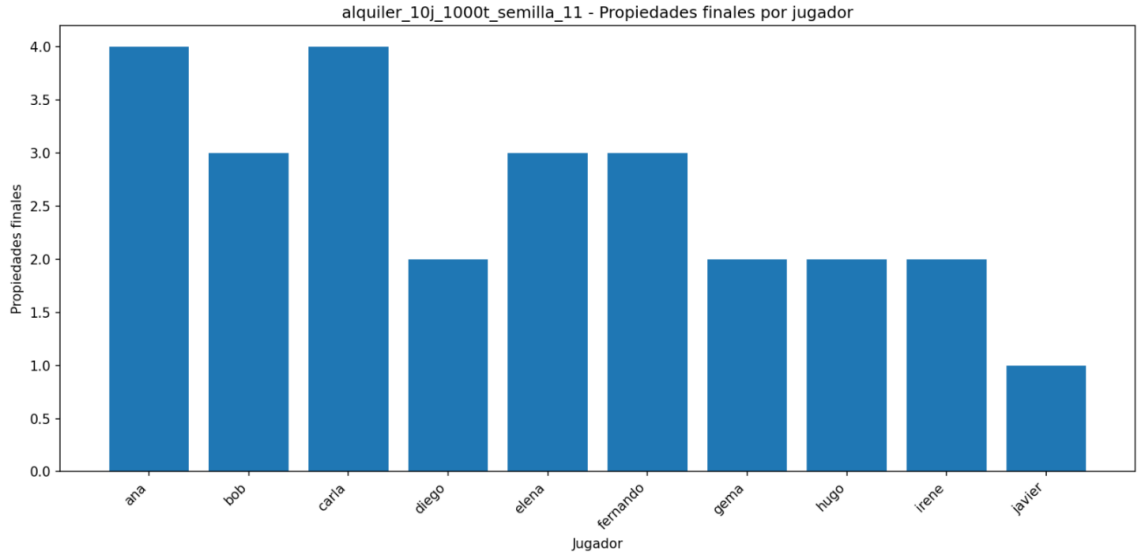
En comparación con la gráfica anterior, aquí el comportamiento es más estable y se ajusta mejor a lo esperado teóricamente, lo que refuerza que, con más simulaciones, la pseudoaleatoriedad se aproxima más al comportamiento real de los dados.

Compras:



En esta gráfica se muestra cómo evolucionan las compras a lo largo de los turnos. Se observa que la mayoría de las compras se concentran al inicio de la partida, cuando las propiedades están disponibles y los jugadores tienen suficiente dinero.

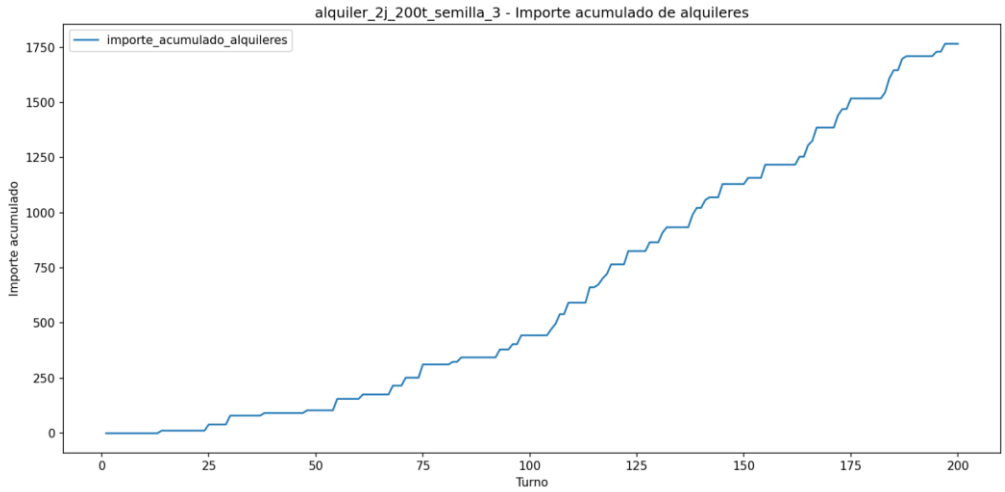
A medida que avanzan los turnos, el número de compras se estabiliza, ya que la mayoría de las propiedades ya han sido adquiridas. Esto refleja un comportamiento esperado dentro del juego, donde las oportunidades de compra disminuyen con el tiempo.



La gráfica muestra una distribución bastante equilibrada de propiedades entre los jugadores, con la mayoría entre 2 y 4.

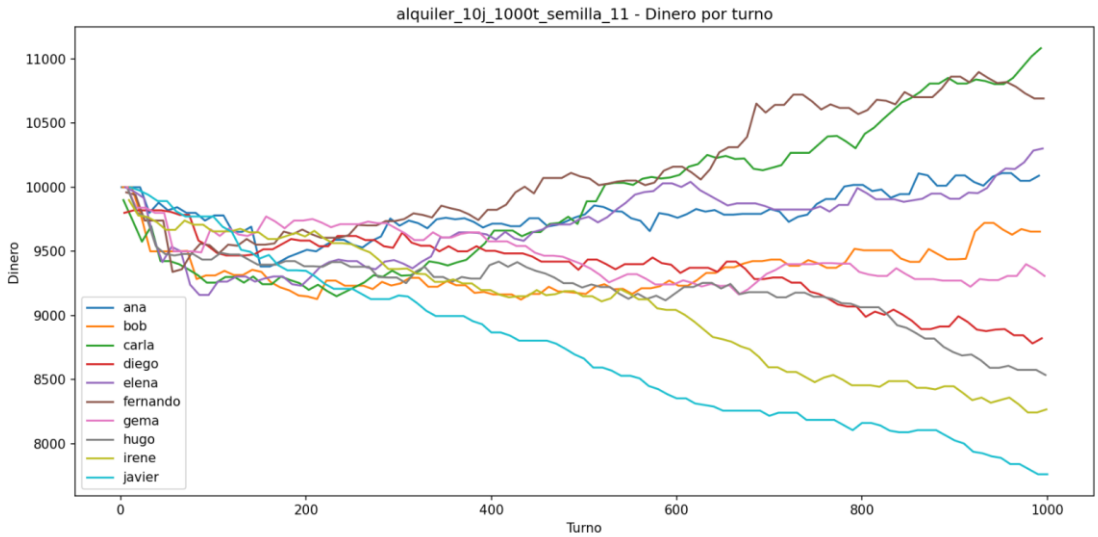
Como solo existe compra al banco, las diferencias se deben principalmente al azar: algunos jugadores han tenido más suerte al caer en casillas comprables que otros.

Alquileres:



La gráfica muestra un crecimiento progresivo del importe acumulado de alquileres a lo largo de los turnos. Al principio apenas hay pagos, pero conforme avanza la partida y los jugadores adquieren más propiedades, los alquileres empiezan a aparecer con mayor frecuencia.

Se observa que el crecimiento no es totalmente uniforme, sino por "saltos", lo que indica momentos en los que se producen pagos importantes. En conjunto, refleja cómo la economía del juego va ganando peso con el tiempo.

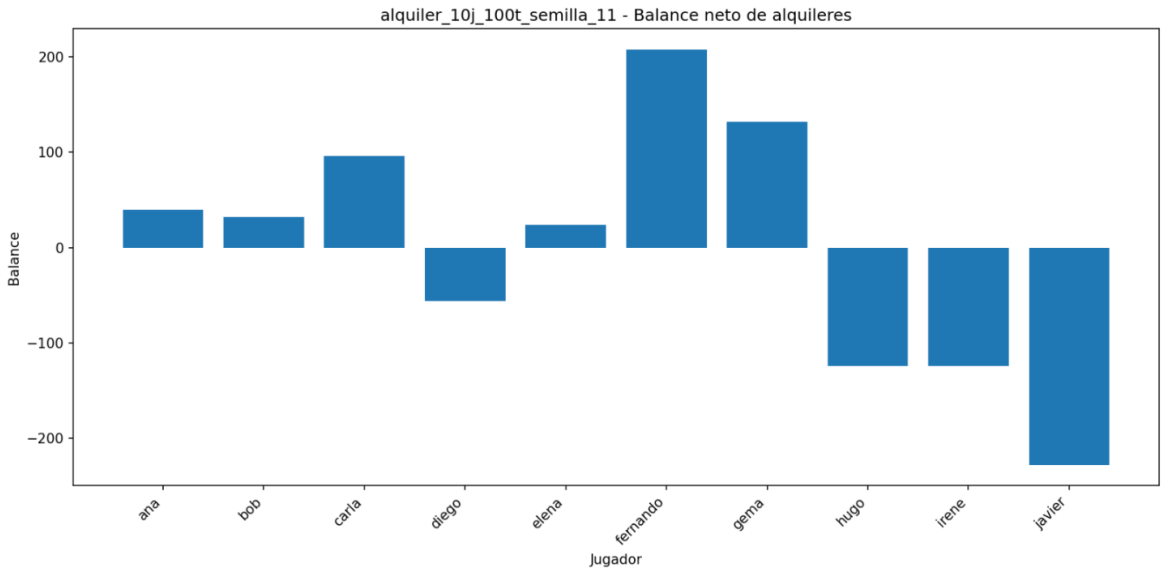


Al inicio de la partida todos los jugadores están bastante igualados y no se distingue

claramente quién va mejor o peor, ya que las diferencias son pequeñas y cambian con frecuencia.

Sin embargo, a partir de unos 200 turnos empiezan a aparecer tendencias claras: los jugadores que van mejor económicamente mantienen e incluso aumentan su ventaja, mientras que los que van peor tienden a perder dinero de forma progresiva.

Esto refleja cómo, a medida que avanza la partida, las diferencias se consolidan y el juego se vuelve más desigual.



La gráfica muestra el balance neto de alquileres por jugador, es decir, quién ha ganado más dinero cobrando alquileres y quién ha perdido más pagando.

Se ve bastante claro que algunos jugadores salen beneficiados (como ernando o gema), mientras que otros acumulan pérdidas importantes. En particular, destaca el caso de javier, que presenta el peor balance, lo que encaja con la gráfica anterior donde se veía que su dinero iba cayendo de forma constante. Esto refleja muy bien cómo los alquileres son el principal factor que acaba marcando las diferencias entre jugadores a lo largo de la partida.

Cárcel:

48	2	bob	encarcelado(0)	tirar
49	1	ana	libre	no_aplica
50	2	bob	encarcelado(1)	tirar
51	1	ana	libre	no_aplica
52	2	bob	encarcelado(2)	tirar
53	1	ana	libre	no_aplica
54	2	bob	encarcelado(3)	liberacion_automatica

La tabla muestra la evolución de un jugador en la cárcel a lo largo de varios turnos. En este caso, Bob permanece encarcelado durante varios turnos consecutivos, intentando salir mediante tiradas de dados sin éxito.

Se observa cómo el número de intentos va aumentando hasta llegar al límite, momento en el que se produce la liberación automática. Mientras tanto, el otro jugador (Ana) continúa jugando con normalidad.

9	1	ana	encarcelado(0)	pagar
10	2	bob	libre	no_aplica
11	1	ana	libre	no_aplica

Se muestra en el caso de Ana que si paga al siguiente turno puede jugar.

87	1	ana	encarcelado(0)	tirar
88	2	bob	encarcelado(0)	tirar
89	1	ana	encarcelado(1)	tirar
90	2	bob	encarcelado(1)	tirar
91	1	ana	libre	no_aplica
92	2	bob	libre	no_aplica

La tabla muestra un caso en el que ambos jugadores entran en la cárcel casi al mismo tiempo y van intentando salir mediante tiradas de dados.

Se observa cómo tanto Ana como Bob fallan en sus primeros intentos, acumulando intentos de salida, hasta que finalmente ambos consiguen salir sacando dobles, sin tener que pagar o quedar libres automáticamente.

Bancarrota:

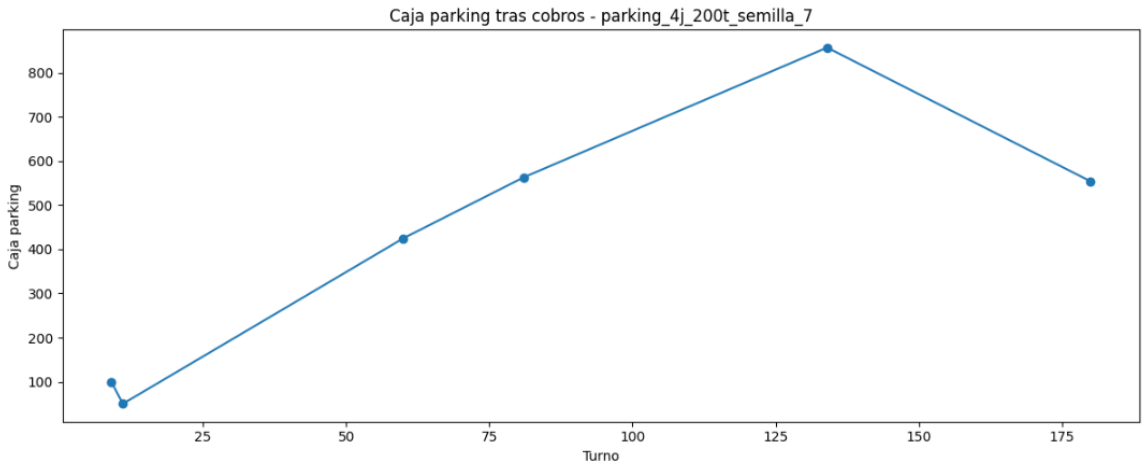
turno	jugador	causa	dinero_antes	valor_liquidacion	dinero_despues	propiedades_liquidadas	resultado
350	javier	alquiler	-12	0	-12	0	eliminado
396	irene	impuesto	-176	110	-66	2	eliminado
414	fernando	alquiler	-12	100	88	1	sobrevive
438	fernando	alquiler	-22	0	-22	0	eliminado
489	gema	alquiler	-32	150	118	2	sobrevive

La tabla muestra los eventos de bancarrota durante la partida, indicando la causa, el dinero antes y después, y si el jugador logra recuperarse o es eliminado.

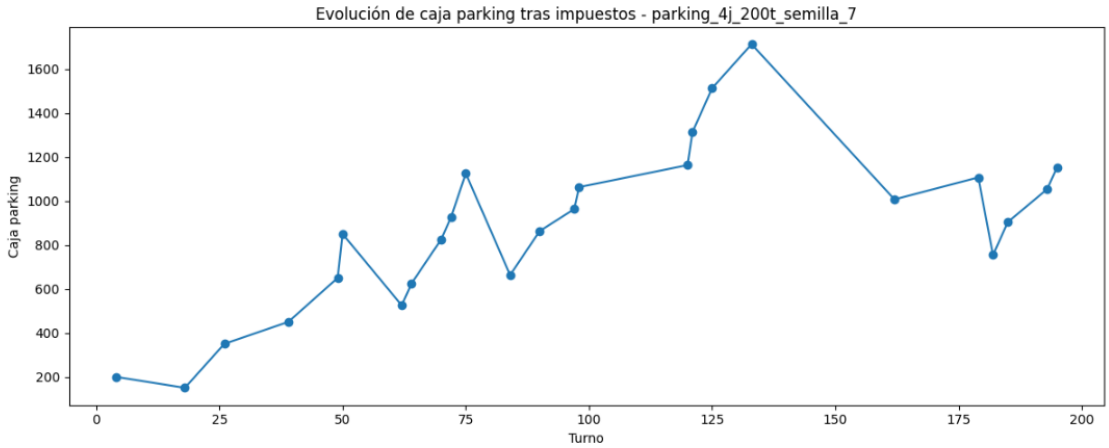
Se observa que la mayoría de bancarrotas se producen por alquileres, lo que confirma su impacto en la economía del juego. Algunos jugadores consiguen recuperarse gracias a la liquidación de propiedades (como Fernando o Gema), mientras que otros no logran salir de números negativos y son eliminados, como Javier o Irene.

En conjunto, refleja bien cómo los alquileres acaban siendo el principal factor de eliminación de jugadores.

Parking:



En esta gráfica se observa cómo varía la caja del parking cuando se producen cobros. Se aprecia que el valor aumenta progresivamente hasta alcanzar un máximo, y posteriormente desciende cuando se reparte el dinero. Esto refleja el funcionamiento del parking como acumulador y posterior redistribuidor de dinero.



En esta gráfica se muestra la evolución completa de la caja del parking, incluyendo tanto los ingresos (principalmente impuestos) como los pagos. Se observa un comportamiento más irregular, con subidas cuando se acumula dinero y bajadas cuando algún jugador cae en parking y recibe el premio. Esto confirma que el sistema introduce variabilidad en la economía del juego.

Monopoly:

turno	jugador	colores_monopolio	casas_compradas	coste_total	dinero_antes	dinero_despues
24	bob	[azul]	2	200	8080	7880
26	bob	[azul]	2	200	7880	7680
28	bob	[azul]	2	200	7680	7480
31	ana	[verde]	3	300	7840	7540
33	ana	[verde]	3	300	7440	7140
35	ana	[verde]	3	300	7184	6884

Al forzar la compra de propiedades, los jugadores acaban formando monopolios, lo que les permite empezar a construir casas.

En la tabla se observa cómo, una vez que un jugador completa un color (por ejemplo, Bob con azul o ana con verde), comienza a comprar casas de forma progresiva en turnos sucesivos. Cada compra reduce su dinero, pero refuerza sus propiedades.

Esto muestra claramente cómo la mecánica de compras facilita la aparición de monopolios y activa la fase de construcción dentro de la partida.